

Analysis Report

Like

Share

839 people like this. [Sign Up](#) to see what your friends like.

Share this Report

GC log file: **gc-2017_07_27-13_58.log.0**

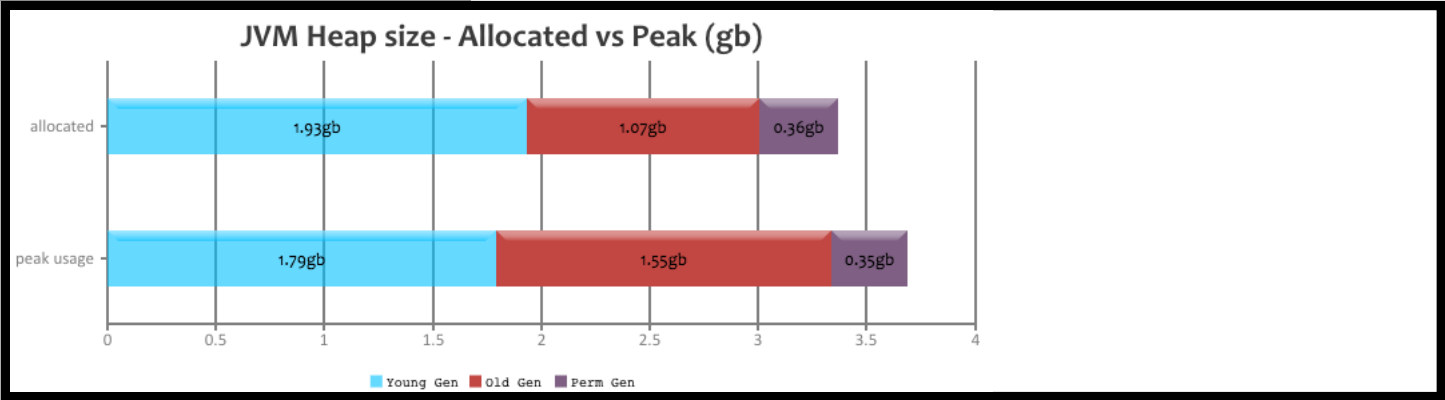
Duration: **1 hr 3 min 55 sec**



Congratulations! Your application's GC activity is healthy.

JVM Heap Size

Generation	Allocated ?	Peak ?
Young Generation	1.93 gb	1.79 gb
Old Generation	1.07 gb	1.55 gb
Perm Generation	372 mb	358.91 mb
Young + Old + Perm	3.36 gb	2.81 gb



Key Performance Indicators

(Important section of the report. To learn more about KPIs, [click here](https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/) (https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/))

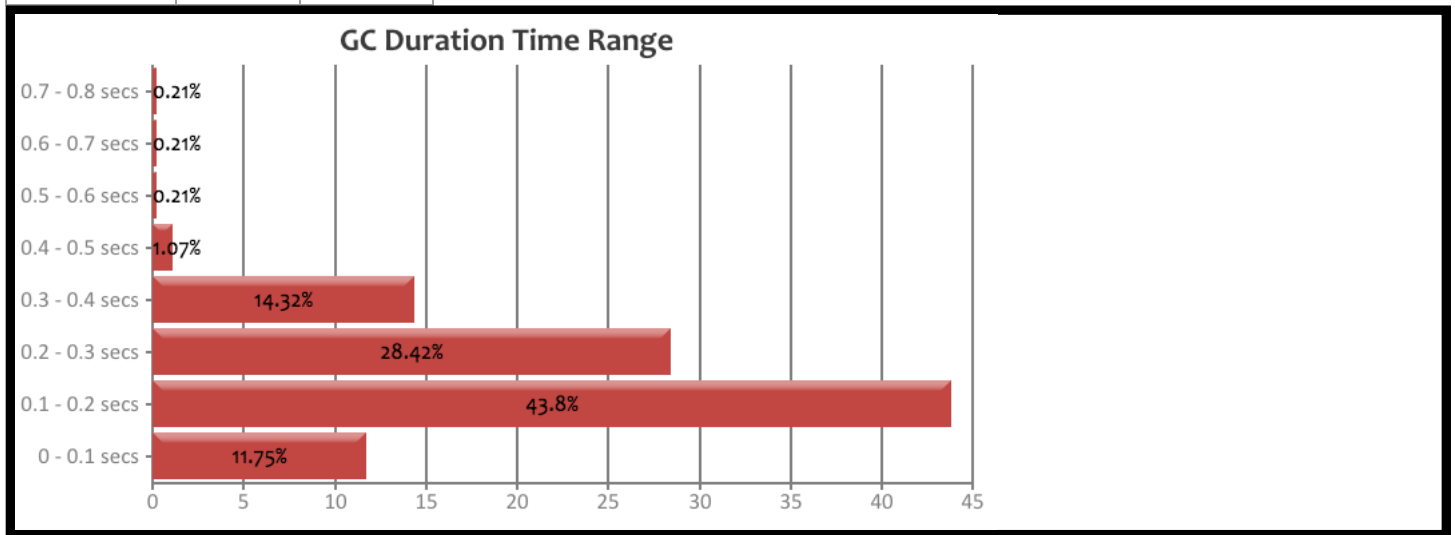
1 **Throughput ?** : 97.502%

2 **Latency:**

Avg GC Time ?	196 ms
Max GC Time ?	800 ms

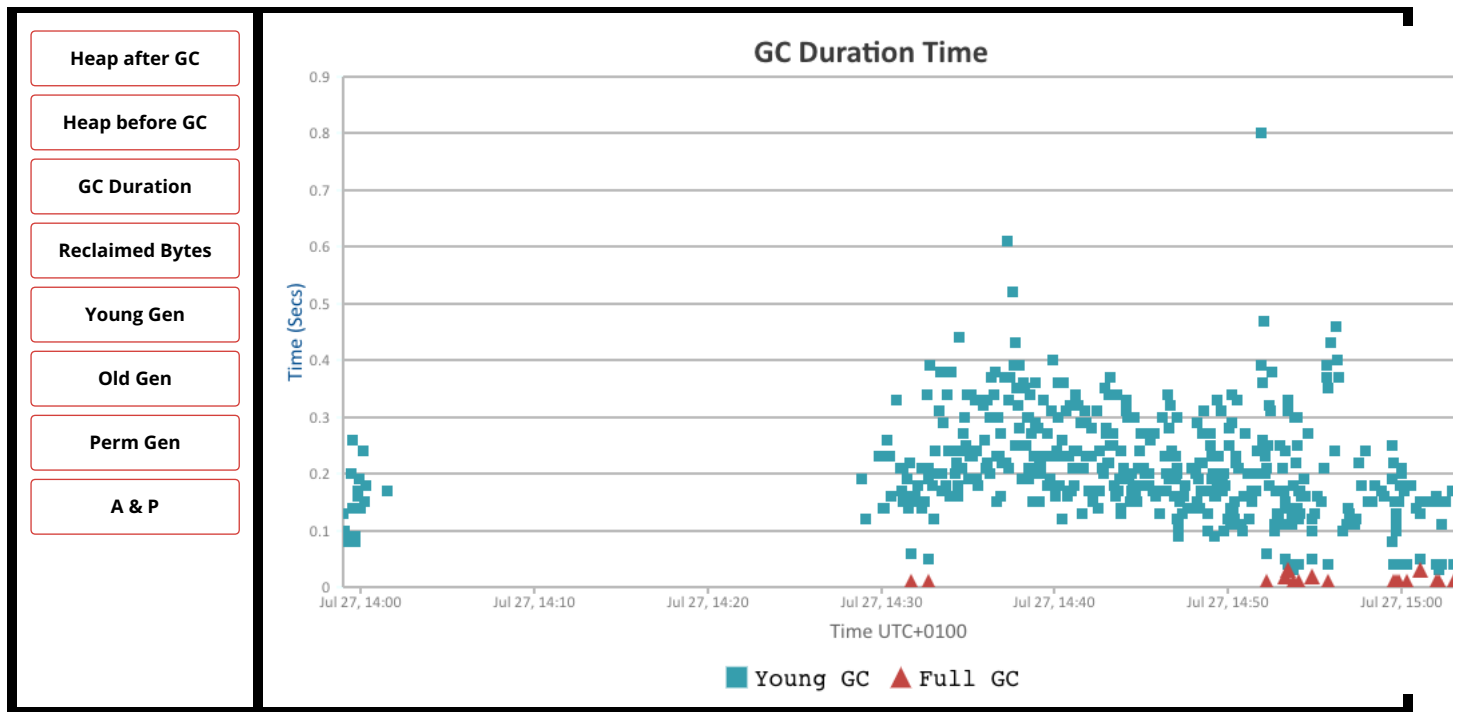
GC Duration Time Range ?:

Duration (secs)	No. of GCs	Percentage
0 - 0.1	55	11.752%
0.1 - 0.2	205	55.556%
0.2 - 0.3	133	83.974%
0.3 - 0.4	67	98.291%
0.4 - 0.5	5	99.359%
0.5 - 0.6	1	99.573%
0.6 - 0.7	1	99.786%
0.7 - 0.8	1	100.0%

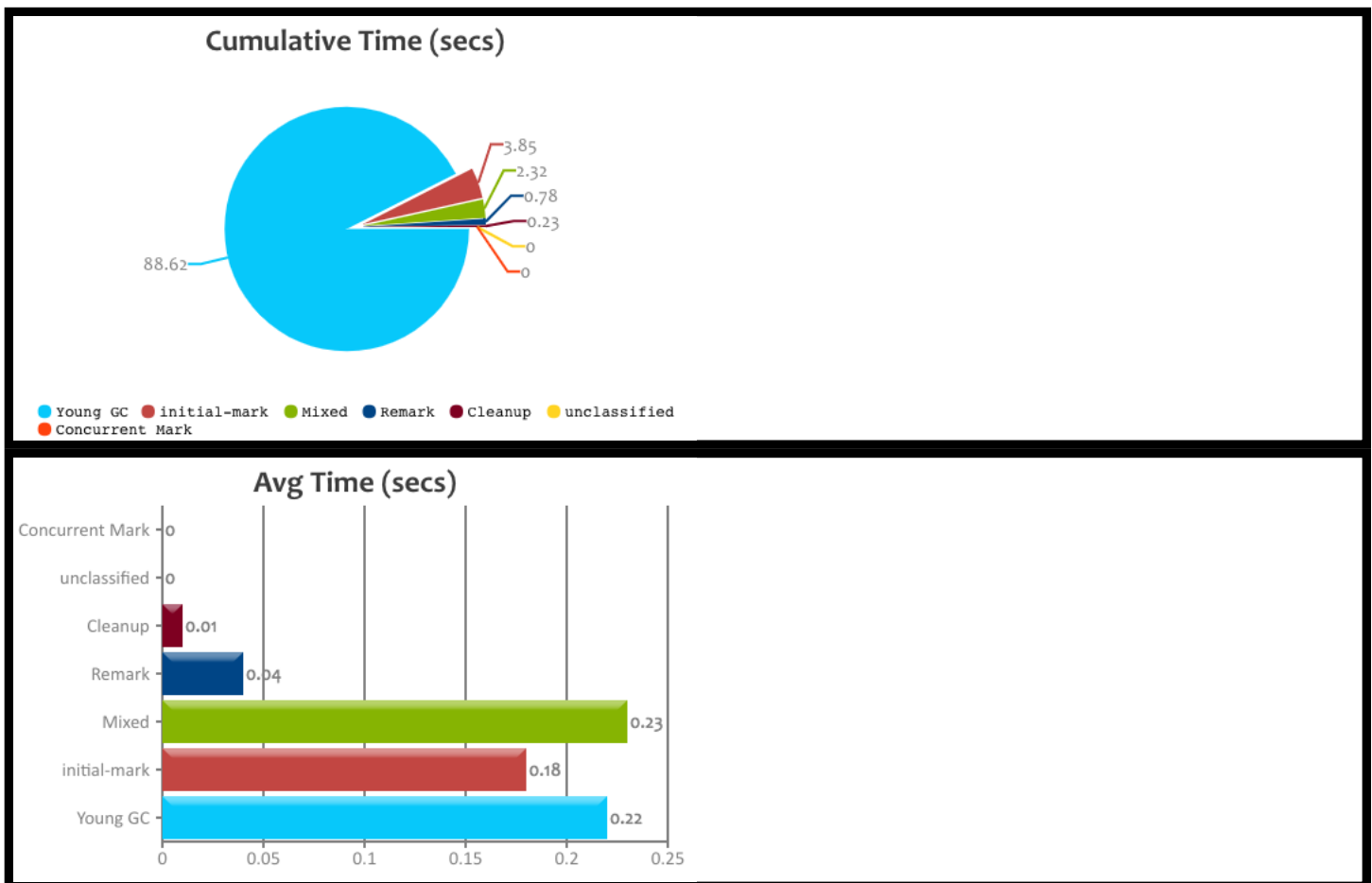


Interactive Graphs

(All graphs are zoomable)



🔍 G1 GC Statistics



	unclassified	Concurrent Mark	Remark	Young GC	Cleanup	Mixed	initial-mark	Total

Count ⓘ	1	18	18	402	18	10	21	488
Total GC Time ⓘ	0	0	780 ms	1 min 28 sec 620 ms	230 ms	2 sec 320 ms	3 sec 850 ms	1 min 35 sec 800 ms
Avg GC Time ⓘ	0	0	43 ms	220 ms	13 ms	232 ms	183 ms	196 ms
Avg Time std dev	0	0	8 ms	87 ms	7 ms	98 ms	71 ms	104 ms
Min/Max Time ⓘ	0 / 0	0 / 0	0 / 60 ms	0 / 800 ms	0 / 30 ms	0 / 460 ms	0 / 390 ms	0 / 800 ms
Avg Interval Time ⓘ	n/a	1 min 50 sec 154 ms	1 min 50 sec 154 ms	9 sec 493 ms	1 min 50 sec 153 ms	58 sec 84 ms	1 min 33 sec 686 ms	24 sec 576 ms

⚙ Object Stats

(These are good metrics to include in your performance reports)

Total created bytes ⓘ	279.69 gb
Total promoted bytes ⓘ	3.63 gb
Avg creation rate ⓘ	74.67 mb/sec
Avg promotion rate ⓘ	991 kb/sec

💧 Memory Leak ⓘ

No major memory leaks.

(**Note:** there are 8 flavours of OutOfMemoryErrors (<https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf>). With GC Logs you can diagnose only 5 flavours of them(java heap space, GC overhead limit exceeded, Requested array size exceeds VM limit, Permgen space, Metaspace). So in other words, your application could be still suffering from memory leaks, but need other tools to diagnose them, not just GC Logs.)

⏴ Consecutive Full GC ⓘ

None.

⏴ Long Pause ⓘ

None.

⌚ Safe Point Duration ⓘ

(To learn more about SafePoint duration, [click here](https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/) (<https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/>))

Not Reported in the log.